

Test Project 2: File Upload & Dattaba

COMP6695001 - Secure Programming

Take Home Assignment

Created by: D7207 - Sandyka Bala

Basic Instructions

Instructions

- Complete the source code given, based on "TODO" instructions in given template.
- Understand what you have implemented is a must, cheating is prohibited. Lecturer has rights to zeroing (0) your score if you cannot obey this rule.

Story

A few weeks after successfully passing Chris's initial skill assessment, your progress at ChainProject co. has earned you greater responsibilities. Chris was impressed with your preliminary authentication flow, but storing temporary data solely in PHP sessions is no longer sufficient for production-grade applications.

Today, ChainProject co. officially kicked off Phase 2 of their enterprise solution architecture. Chris approaches your desk with an upgraded set of requirements: you need to migrate the temporary session-based storage into a persistent database and implement a secure profile picture upload feature. Since secure programming is paramount, Chris specifically stresses that uploaded files must be strictly validated to prevent Remote Code Execution (RCE) and directory traversal vulnerabilities.

Your goal in this episode is to **refactor and expand the application** according to the specifications in the "**Technical Instructions**". Show Chris that you are ready to build robust, secure backend systems for real-world clients!

<https://github.com/HeyThisHaku/UploadHere.git>.

Now, your job is to **complete the source code** listed in "**Technical Instructions**", to show that you are ready to receive real projects from industry. When the deadline reaches, you should **submit the completed project files** contains required implementations listed in "Technical Instructions". Have a look at the next page to see what you should complete!

Technical Instructions

Registration Page - 25 points

- The registration page is designed to store new user data in the database once the input meets all validation requirements.
- The registration process is split into two files: `register.php` (the UI) and `doRegister.php` (the logic for registering new users).
- The registration page consists of a form with input fields for the user's username, email address, password, and confirm password.
- **[5 points]** In `register.php`, ensure that when the user submits the registration form (by clicking the Register button), the input data is sent to `doRegister.php` using an HTTP POST request.
- **[20 points]** When input is received by `doRegister.php`, the program will execute the following logic:
 - **[12 points]** Validate the data entered by the user against the following rules:
 - * **[3 points]** Username, validate the input must not be empty.
 - * **[3 points]** Email address, validate the input must be a valid email format.
 - * **[3 points]** Uniqueness, ensure that the username and email do not already exist in the database.
 - * **[3 points]** Password, validate the input must be at least 8 characters long, and must perfectly match the Confirm Password.
 - **[3 points]** If any of the validations fail, redirect the user back to `register.php` and display an error message indicating the specific problem.
 - **[3 points]** If all validations pass, securely hash the password (e.g., using `password_hash()`).
 - **[2 points]** Save the new user data (username, email, hashed password) into the `users` table in the database and redirect the user to `login.php`.

Login Page - 25 points

- The login page is designed to authenticate the user by verifying that the input matches the credentials stored in the database.
- The login process is split into two files: `login.php` (the UI) and `doLogin.php` (the logic for logging in users).
- The login page consists of a form with input fields for the user's username and password.
- **[5 points]** In `login.php`, ensure that when the user submits the login form, the input data is sent to `doLogin.php` using an HTTP POST request.
- **[20 points]** When input is received by `doLogin.php`, the program will execute the following logic:

- [12 points] Query the `users` table using the provided username and compare the input password against the securely hashed password stored in the database.
- [5 points] If the credentials match, save the authenticated user data (such as user ID and username) to the Session and redirect the user to `index.php`.
- [3 points] If no matching user is found or the password is incorrect, redirect the user back to `login.php` and display an error message (e.g., "Wrong username or password combination").

Upload Page - 30 points

- The upload page is designed to allow authenticated users to securely upload files to the server.
- The upload process is split into two files: `upload.php` (the UI) and `doUpload.php` (the logic for handling file uploads).
- The upload page consists of a form with a file input field.
- [5 points] In `upload.php`, ensure that the form uses an HTTP POST request and includes the `enctype="multipart/form-data"` attribute to handle file transfers.
- [25 points] When a file is received by `doUpload.php`, the program will execute the following logic:
 - [12 points] Validate the request against the following rules:
 - * [4 points] Session validation, ensure only authenticated users can upload files.
 - * [4 points] File size, limit the uploaded file to a maximum of 5MB.
 - * [4 points] File type, only allow specific extensions (e.g., PDF, DOCX, JPG, PNG).
 - [3 points] If any of the validations fail, redirect the user back to `upload.php` with an error message indicating the problem.
 - [5 points] If all validations pass, move the uploaded file to a designated physical directory on the server (the `uploads/` folder).
 - [5 points] Insert the file metadata (original file name, stored file name, file size, file type, and the uploader's `user_id` from the Session) into the `files` table in the database, then redirect back to `list.php`.

Home & List Page - 20 points

- The list page is designed to display a table of all files that have been uploaded by the currently authenticated user.
- [5 points] In `list.php`, validate the user session. If an unauthenticated user attempts to access the page, immediately redirect them to `login.php`.

- **[10 points]** Query the `files` table to fetch the file records belonging specifically to the logged-in user (using their `user_id` from the Session) and display them dynamically in the HTML table.
- **[5 points]** Create Button Logout & Implement a Logout functionality. When the user clicks Logout on `index.php`, completely clear the authenticated user's Session data (`session_destroy()`) and redirect them to `login.php`.

- GLHF